ReactJS

Unit 1 — Introduction

What is React · Get started · Project structure · Render HTML

In This Unit

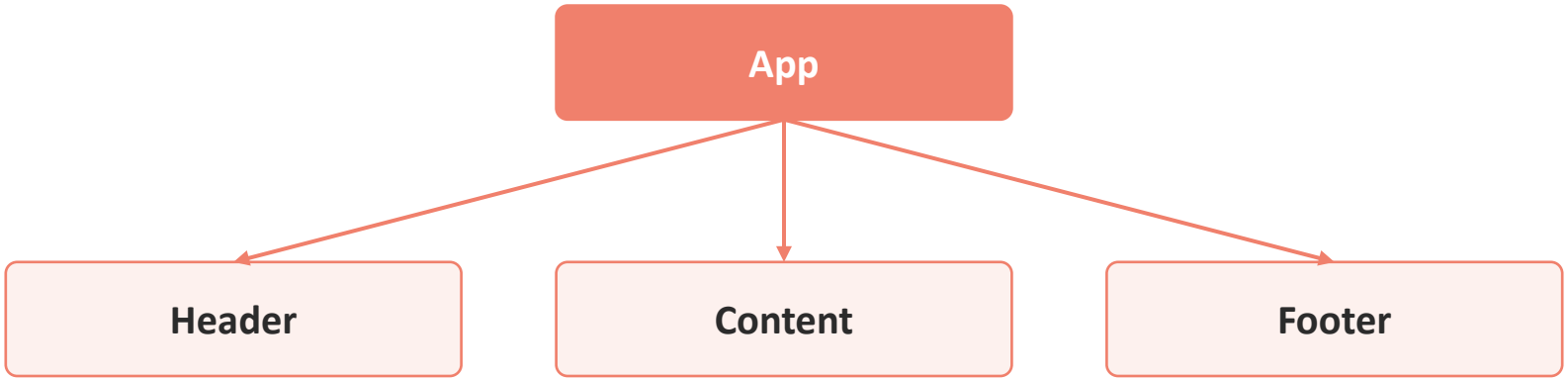
1. What is React?
2. How does React work?
3. Features and React vs JavaScript
4. Single Page Applications
5. Get started and project structure
6. Render HTML

What is React?

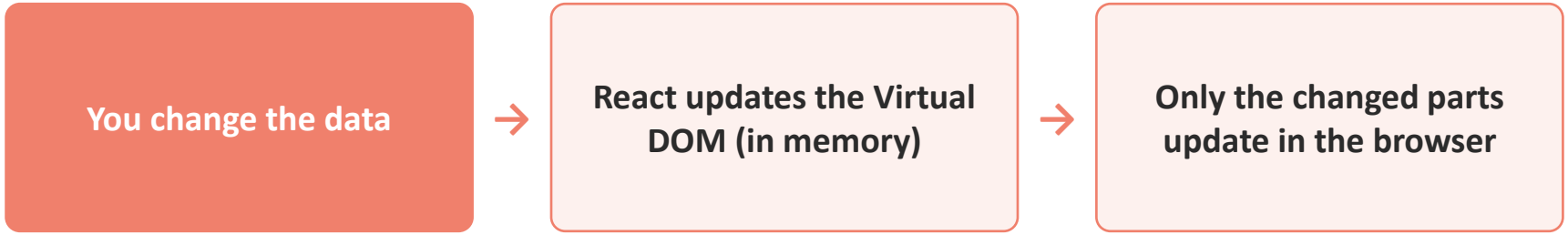
- React is a front-end JavaScript library
- It is used to build user interfaces (UI)
- It is also called React.js or ReactJS
- Created at Facebook (now Meta); first used in 2011 for the Facebook Newsfeed
- Used to build Single Page Applications

What is a Component?

- A React app is built from small pieces called components
- Each component is a reusable part of the page



How Does React Work?



- Virtual DOM = a copy of the page that React keeps in memory
- React compares old and new, then changes only what changed

Features of React

Components

Build the page from small, reusable parts.

JSX

Write HTML-like code inside JavaScript.

Declarative

Describe what the page should look like.

Fast updates

Virtual DOM changes only what is needed.

React vs Plain JavaScript

Plain JavaScript

```
document.getElementById("msg")
  .innerHTML = "Hello";
```

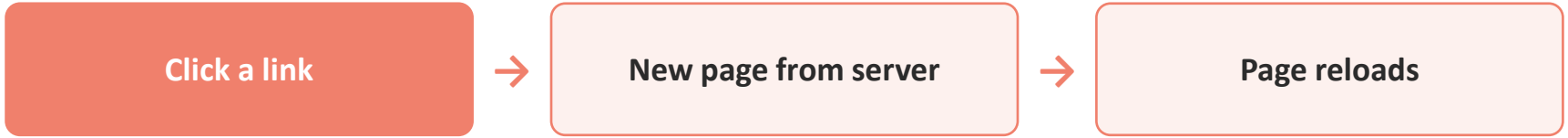
React

```
<h2>{message}</h2>
```

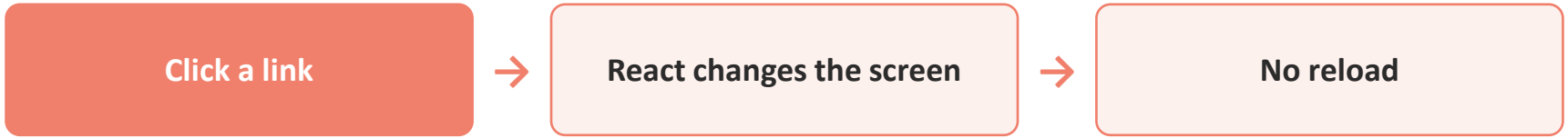
- Plain JavaScript: you find the element and change it
- React: you describe the page; React updates it when data changes

Single Page Application (SPA)

Normal website



React SPA



A SPA loads one HTML page and updates it with JavaScript.

Get Started: Create a React App

- Install Node.js first
- Create the app with Vite, then run it

Terminal

```
npm create vite@latest my-react-app -- --template react
cd my-react-app
npm install
npm run dev
```

Open the address shown in the terminal (usually http://localhost:5173). Older tutorials use create-react-app, which is no longer recommended.

Project Structure

```
my-react-app/
|-- node_modules/
|-- public/
|-- src/
|     |-- App.jsx
|     +-- main.jsx
|-- index.html
+-- package.json
```

File / Folder	What it does
node_modules/	Installed libraries
public/	Static files (images)
src/	Your React code
App.jsx	Main component
main.jsx	Starts the app
index.html	The one HTML page
package.json	Project settings

How the Files Connect



main.jsx

```
import { createRoot } from 'react-dom/client'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <App />
)
```

Render HTML: createRoot and render

- React shows HTML on the page using createRoot() and render()
- createRoot() takes one HTML element: the container
- render() says what to show inside it

```
createRoot(document.getElementById('root')).render(
  <h1>Hello React!</h1>
)
```

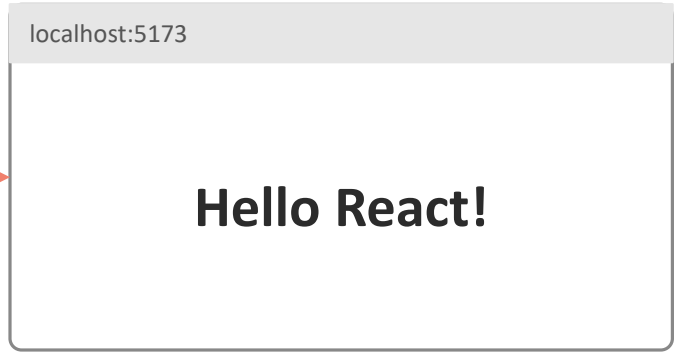
Example: Hello React!

Example

```
import { createRoot }
  from 'react-dom/client'

createRoot(
  document.getElementById('root')
).render(
  <h1>Hello React!</h1>
)
```

Result



The <h1> appears inside .

The Root Node

- The root node is the HTML element where React shows the result
- It is the container for everything React manages
- The standard id is root, but you can use any name

```
<div id="root"></div>                // in index.html
```

```
createRoot(document.getElementById('root'))
```

Why Use React?

Reusable

Write a component once, use it many times.

Easy to manage

Small components are easier to fix.

Fast

Only changed parts are updated.

Popular

Large community and many libraries.

Great for SPAs

Smooth pages without reloads.

Practice

- Create a React app with Vite and run it
- Open src/App.jsx and change the text to your name
- Find in index.html
- Find createRoot in main.jsx
- Render an <h1> directly with render() and see the result

Summary

- React is a front-end JavaScript library for building UIs
- Pages are built from reusable components
- React updates only what changed (Virtual DOM)
- React apps are usually Single Page Applications
- createRoot().render() puts React into the root element